练习1：基于x87 FPU优化矩阵乘法性能

- **任务**：利用处理器中的x87 FPU（浮点计算单元）中更多的寄存器对矩阵乘法计算的循环进行展开以提升性能。
- **子任务**：
  - 使用`FLD`、`FSTP`等指令操作数据的读取与存储；
  - 使用`FMUL`、`FADDP`等x87 FPU提供的指令进行计算；
  - 以2为步进对N维度的循环进行展开（可参考`src/lab3/gemm_kernel_baseline.S`中的`DO_LOOP_N`逻辑）
- **目标**：学会利用计算单元的特性进行循环展开。

> **关于浮点指令**
>
> 本实验涉及x87 FPU的通用寄存器栈以及`FLD`、`FSTP`、`FMUL`、`FADDP`等常用指令，相关内容可自行回顾实验1 - 实验原理 - 第3节 x87 FPU。

0. 下载并解压实验包

从首页链接下载`lab3.tar.gz`，将其拷贝至用户目录，并执行命令`tar -zxvf lab3.tar.gz`解压实验包。

1. 在`src/lab3/gemm_kernel_opt_loop_unrolling.S`第83行添加计算`A[m][k]*B[k][n+1]->st(0)`的逻辑

```
81      fmul %st(1), %st(0)                 // 计算A[m][k] * B[k][n]  --> st(0)
82
83      // TODO  练习1: 添加计算A[m][k] * B[k][n+1] --> st(0)的逻辑
```

2. 在`src/lab3/gemm_kernel_opt_loop_unrolling.S`第91行添加加载`C[m][n]->st(1)`和加载`C[m][n+1]->st(0)`的逻辑

```
91      // TODO 练习1: 请添加加载C[m][n] --> st(1) 和 C[m][n+1] --> st(0)的逻辑
```

3. 在`src/lab3/gemm_kernel_opt_loop_unrolling.S`第93行添加部分和累加逻辑：`C[m][n+1] + A[m][k] * B[k][n+1]`和`C[m][n] + A[m][k] * B[k][n]`

Processing math: 100%

```
93      // TODO 练习1: 请添加部分和累加逻辑: C[m][n+1] + A[m][k] * B[k][n+1] 和 C[m][n] + A[m][k] * B[k][n]
```

4. 在 `src/lab3/gemm_kernel_opt_loop_unrolling.S` 第97行添加保存 `C[m][n]` 的逻辑

```
97      // TODO 练习1: 请添加保存C[m][n]的逻辑
```

5. 在 `src/lab3/gemm_kernel_opt_loop_unrolling.S` 第99行添加N维度的循环更新逻辑

```
99      // TODO 练习1: 请添加N维度的循环更新逻辑
```

6. 验证循环展开方案的正确性

```
# 项目根目录下执行
mkdir -p build && cd build
cmake -B . -S ../ && cmake --build ./ --target
lab3_gemm_opt_loop_unrolling.unittest
./dist/bins/lab3_gemm_opt_loop_unrolling.unittest
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_loop_unrolling.unittest
Running main() from /home/lenovo/test1/TA/try/computer_archtecture_labs/build/_d
eps/googletest-src/googletest/src/gtest_main.cc
[==========] Running 3 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 3 tests from gemm_kernel_opt_loop_unrolling
[ RUN      ] gemm_kernel_opt_loop_unrolling.test0
[       OK ] gemm_kernel_opt_loop_unrolling.test0 (1 ms)
[ RUN      ] gemm_kernel_opt_loop_unrolling.test1
[       OK ] gemm_kernel_opt_loop_unrolling.test1 (0 ms)
[ RUN      ] gemm_kernel_opt_loop_unrolling.test2
[       OK ] gemm_kernel_opt_loop_unrolling.test2 (1 ms)
[----------] 3 tests from gemm_kernel_opt_loop_unrolling (3 ms total)

[----------] Global test environment tear-down
[==========] 3 tests from 1 test suite ran. (3 ms total)
[  PASSED  ] 3 tests.
```

7. 对比优化后的算法与基线的性能（测试用例根据自己的优化选择），根据对比结果分析优化方案的局限性

```
# build目录下执行
cmake -B . -S ../ && cmake --build ./ --target lab3_gemm_opt_loop_unrolling
./dist/bins/lab3_gemm_opt_loop_unrolling 256 256 256
```

Processing math: 100%

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab3_gemm_
opt_loop_unrolling 256 256 256
--- Performance before loop unrolling optimization ---
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 3.51561
                Cost(s): 0.017578
                Benchmark(Gflops): 1.90888
--- Performance for after loop unrolling optimization ---
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 2.60652
                Cost(s): 0.0130326
                Benchmark(Gflops): 2.57465
----------------------------
Performance difference(Gflops): 0.665766
```

8. 测试并对比不同循环展开度的性能差异

循环展开度指的是在每轮循环中，原始循环体被重复执行的次数。

测试不同循环展开度（比如0（即无循环展开）、2、4等）的性能差异，绘制性能变化曲线图并进行合理分析。

Processing math: 100%